# WebSeed 지원 분석 문서

## 개요

WebSeed는 BEP 19에 정의된 BitTorrent 확장으로, HTTP/HTTPS 서버를 통해 토렌트 데이터를 다운로드할 수 있게 해줍니다. Rain BitTorrent 클라이언트는 완전한 WebSeed 구현을 제공하여 일반 피어와 HTTP 서버를 통합된 방식으로 활용할 수 있도록 합니다.

### 주요 특징

- BEP 19 WebSeed 프로토콜 완전 구현
- HTTP Range 요청을 통한 효율적 다운로드
- 일반 피어와의 완전한 통합 (피어가 WebSeed에서 피스를 가져올 수 있음)
- 자동 실패 처리 및 재시도 메커니즘
- 속도 제한 및 동시 다운로드 제어
- 단일 파일과 다중 파일 토렌트 모두 지원

## HTTP 기반 시드 구현

### 1. WebSeed 소스 구조

각 WebSeed URL은 개별 소스로 관리됩니다 ( `internal/webseedsource/webseedsource.go` ):

```
// WebseedSource는 웹 소스에서 토렌트 데이터를 다운로드하기 위한 URL입니다.
type WebseedSource struct {
URL  string  // WebSeed URL
```

```go
Disabled bool // 비활성화 여부
Downloader *urldownloader.URLDownloader // 현재 다운로더
LastError error // 마지막 오류
DisabledAt time.Time // 비활성화 시간
DownloadSpeed metrics.Meter // 다운로드 속도 측정
}

// WebSeed 소스 목록 생성
func NewList(sources []string) []*WebseedSource {
l := make([]*WebseedSource, len(sources))
for i := range sources {
l[i] = &WebseedSource{
URL: sources[i],
DownloadSpeed: metrics.NilMeter{},
}
}
return l
}

// 현재 다운로드 중인지 확인
func (s *WebseedSource) Downloading() bool {
return s.Downloader != nil
}
```

```go
// 남은 피스 수 반환
func (s *WebseedSource) Remaining() uint32 {
if s.Downloader == nil {
return 0
}
return s.Downloader.End - s.Downloader.ReadCurrent() - 1
}
```

## 2. URL 다운로더 구현

HTTP 요청을 통한 실제 다운로드를 담당합니다 ( `internal/urldownloader/urldownloader.go` ):

```go
// URLDownloader는 HTTP 소스에서 파일을 다운로드합니다.
type URLDownloader struct {
URL string // 다운로드 URL
Begin, End, current uint32 // 피스 인덱스 범위
bucket *ratelimit.Bucket // 속도 제한
closeC, doneC chan struct{} // 종료 채널
}

// 다운로드 결과 구조체
type PieceResult struct {
Downloader *URLDownloader // 다운로더 참조
```

```
Buffer bufferpool.Buffer // 다운로드된 데이터

Index  uint32  // 피스 인덱스

Error  error  // 오류 정보

Done  bool  // 모든 피스 다운로드 완료 여부

}



// 새 URL 다운로더 생성

func  New(source  string, begin, end  uint32, b *ratelimit.Bucket) *URLDownloade

return &URLDownloader{

URL: source,

Begin: begin,

current: begin,

End: end,

bucket: b,

closeC: make(chan  struct{}),

doneC: make(chan  struct{}),

}

}
```

## 3. HTTP Range 요청 처리

WebSeed는 HTTP Range 요청을 사용하여 필요한 부분만 다운로드합니다:

```
func (d *URLDownloader) Run(client *http.Client, pieces []piece.Piece, multifile
```

```go
resultC  chan *PieceResult, pool *bufferpool.Pool, readTimeout time.Duration) {
// 다운로드 작업 생성
jobs := createJobs(pieces, d.Begin, d.readEnd())
processJob := func(job downloadJob) bool {
// URL 구성 (단일/다중 파일 지원)
u := d.getURL(job.Filename, multifile)
// HTTP GET 요청 생성
req, err := http.NewRequest(http.MethodGet, u, nil)
if  err != nil {
d.sendResult(resultC, &PieceResult{Downloader: d, Error: err})
return false
}
// Range 헤더 설정 (바이트 범위 지정)
req.Header.Set("Range", fmt.Sprintf("bytes=%d-%d",
job.RangeBegin, job.RangeBegin+job.Length-1))
// 요청 실행
resp, err := client.Do(req)
if  err != nil {
d.sendResult(resultC, &PieceResult{Downloader: d, Error: err})
return false
}
defer  resp.Body.Close()
// 상태 코드 확인 (200 OK 또는 206 Partial Content)
```

```go
err = checkStatus(resp)
if err != nil {
d.sendResult(resultC, &PieceResult{Downloader: d, Error: err})
return false
}
// 데이터 읽기 (속도 제한 적용)
// ... 읽기 로직 ...
}
}

// 상태 코드 검증
func checkStatus(resp *http.Response) error {
switch resp.StatusCode {
case 200, 206: // OK 또는 Partial Content
return nil
default:
return fmt.Errorf("unexpected status code: %d", resp.StatusCode)
}
}
```

## WebSeed와 일반 피어의 통합

### 1. 피스 피커 통합

WebSeed는 피스 피커에 완전히 통합되어 일반 피어와 함께 관리됩니다 (`internal/piecepicker/webseed.go`):

```go
// PiecePicker 구조체에 WebSeed 소스 포함
type PiecePicker struct {
webseedSources []*webseedsource.WebseedSource
pieces []myPiece
maxWebseedPieces int // WebSeed가 한 번에 다운로드할 최대 피스 수
// ...
}

// WebSeed 다운로드 스펙
type WebseedDownloadSpec struct {
Source *webseedsource.WebseedSource
Begin uint32 // 시작 피스 인덱스
End uint32 // 종료 피스 인덱스 (exclusive)
}

// WebSeed용 피스 선택
func (p *PiecePicker) PickWebseed(src *webseedsource.WebseedSource) *WebseedDown
r := p.findPieceRangeForWebseed()
if r == nil {
return nil
}
```

```go
// 선택된 범위를 WebSeed 다운로드 중으로 표시
for i := r.Begin; i < r.End; i++ {
if p.pieces[i].RequestedWebseed != nil {
panic("already downloading from webseed url")
}
p.pieces[i].RequestedWebseed = src
}
return &WebseedDownloadSpec{
Source: src,
Begin: r.Begin,
End: r.End,
}
}
```

## 2. 갭 기반 피스 선택

WebSeed는 연속된 피스 범위(갭)를 우선적으로 선택합니다:

```go
// WebSeed용 피스 범위 찾기
func (p *PiecePicker) findPieceRangeForWebseed() *Range {
gaps := p.findGaps()
if len(gaps) == 0 {
// 갭이 없으면 다른 WebSeed에서 피스를 가져옴
return p.webseedStealsFromAnotherWebseed()
```

```go
}

// 가장 큰 갭 중에서 랜덤 선택

gap := selectRandomLargestGap(gaps)

return &gap

}



// 갭 찾기 (BEP 19 정의에 따름)

func (p *PiecePicker) findGaps() []Range {

a := make([]Range, 0, len(p.pieces)/2)

var inGap bool

var begin uint32

for _, pi := range p.pieces {

if !inGap {

if pi.AvailableForWebseed() {

begin = pi.Index

inGap = true

}

} else {

r := Range{Begin: begin, End: pi.Index}

if !pi.AvailableForWebseed() {

a = append(a, r)

inGap = false

} else if r.Len() == p.maxWebseedPieces {
```

```
// 최대 크기에 도달하면 갭 분할
a = append(a, r)
begin = pi.Index
}
}
}
if inGap {
a = append(a, Range{Begin: begin, End: uint32(len(p.pieces))})
}
return a
}
```

### 3. 피어와 WebSeed 간 경쟁

피어가 WebSeed에서 다운로드 중인 피스를 가져올 수 있습니다:

```
// 피어가 WebSeed에서 피스를 가져오기
func (p *PiecePicker) peerStealsFromWebseed(pe *peer.Peer) *myPiece {
downloading := p.getDownloadingSources()
for _, src := range downloading {
if src.Remaining() == 0 {
continue
}
// 뒤에서부터 검색 (마지막 피스부터)
```

```
for  i := src.Downloader.End - 1;  i > src.Downloader.ReadCurrent();  i-- {
pi := &p.pieces[i]
// 피어가 이 피스를 가지고 있고 요청되지 않았으면
if  pi.Done || pi.Writing {
continue
}
if !pi.Having.Has(pe) {
continue
}
if  pi.Requested.Len() > 0 {
continue
}
// WebSeed 다운로드 중단하고 피어에게 할당
p.WebseedStopAt(src, i)
return  pi
}
}
return  nil
}
```

## 실패 처리 및 폴백 메커니즘

### 1. 오류 처리

WebSeed 다운로드 중 오류가 발생하면 자동으로 처리됩니다 ( `torrent/torrent_webseed.go` ):

```go
func (t *torrent) handleWebseedPieceResult(msg *urldownloader.PieceResult) {
if msg.Error != nil {
t.log.Debugln("webseed download error:", msg.Error)
// 가능한 오류 원인:
// * Client.Do 오류 (네트워크 문제)
// * 예상치 못한 상태 코드
// * Response.Body.Read 오류
t.disableSource(msg.Downloader.URL, msg.Error, true)
t.webseedActiveDownloads--
t.startPieceDownloaders() // 다른 다운로더 시작
return
}
// 성공적인 다운로드 처리
piece := &t.pieces[msg.Index]
t.log.Debugf("piece #%d downloaded from %s", msg.Index, msg.Downloader.URL)
// 통계 업데이트
t.bytesDownloaded.Inc(int64(len(msg.Buffer.Data)))
t.downloadSpeed.Mark(int64(len(msg.Buffer.Data)))
// WebSeed 소스별 속도 통계
for _, src := range t.webseedSources {
if src.URL != msg.Downloader.URL {
```

```
continue
}
src.DownloadSpeed.Mark(int64(len(msg.Buffer.Data)))
break
}
// 피스 쓰기 시작
pw := piecewriter.New(piece, msg.Downloader, msg.Buffer)
go pw.Run(t.pieceWriterResultC, t.doneC, ...)
}
```

## 2. 소스 비활성화 및 재시도

```
func (t *torrent) disableSource(srcurl string, err error, retry bool) {
for _, src := range t.webseedSources {
if src.URL != srcurl {
continue
}
// 소스 비활성화
src.Disabled = true
src.DisabledAt = time.Now()
src.LastError = err
// 현재 다운로더 종료
t.closeWebseedDownloader(src)
```

```
// 재시도 스케줄링
if  retry {
go  t.notifyWebseedRetry(src)
}
break
}
}

// 1분 후 재시도 알림
func (t *torrent) notifyWebseedRetry(src *webseedsource.WebseedSource) {
select {
case <-time.After(time.Minute):
select {
case  t.webseedRetryC <- src: // 재시도 신호
case <-t.closeC:
}
case <-t.closeC:
}
}
```

### 3. 동적 다운로드 범위 조정

WebSeed 다운로드 중에 범위를 동적으로 조정할 수 있습니다:

```go
// 토렌트 쓰기 완료 시 WebSeed 범위 조정
func (t *torrent) handlePieceWritten(pw *piecewriter.PieceWriter) {
// ... 기타 처리 ...
if  t.piecePicker != nil {
_, ok := pw.Source.(*urldownloader.URLDownloader)
src := t.piecePicker.RequestedWebseedSource(pw.Piece.Index)
// 피어가 WebSeed 피스를 완료한 경우
if !ok && src != nil {
closed := t.piecePicker.WebseedStopAt(src, pw.Piece.Index)
if  closed {
t.log.Debugf("closed webseed downloader: %s", src.URL)
t.startPieceDownloaderForWebseed(src) // 새 범위로 재시작
}
}
}
}
```

# 토렌트 시스템 통합

## 1. 토렌트 구조체 통합

WebSeed는 토렌트 구조체에 완전히 통합됩니다 ( `torrent/torrent.go` ):

```go
type torrent struct {
```

```
// ... 기타 필드들 ...
webseedClient *http.Client // HTTP 클라이언트
webseedSources []*webseedsource.WebseedSource // WebSeed 소스 목록
rawWebseedSources []string  // 원본 URL 목록
webseedPieceResultC *suspendchan.Chan[*urldownloader.PieceResult] // 결과 채널
webseedRetryC  chan *webseedsource.WebseedSource // 재시도 채널
webseedActiveDownloads  int  // 활성 다운로드 수
// ... 기타 필드들 ...
}
```

## 2. WebSeed 다운로더 시작

```
func (t *torrent) startPieceDownloaderForWebseed(src *webseedsource.WebseedSourc
if  src.Disabled {
return false
}
if  t.webseedActiveDownloads >= t.session.config.WebseedMaxDownloads {
return false
}
if  t.piecePicker == nil {
return false
}
// WebSeed용 피스 선택
```

```
spec := t.piecePicker.PickWebseed(src)
if  spec == nil {
return false
}
// URL 다운로더 생성
downloader := urldownloader.New(
src.URL,
spec.Begin,
spec.End,
t.session.speedLimitDownload,
)
src.Downloader = downloader
t.webseedActiveDownloads++
// 다운로드 고루틴 시작
go  downloader.Run(
t.webseedClient,
t.pieces,
t.info.IsMultiFile(),
t.webseedPieceResultC.SendC(),
t.session.bufferPool,
t.session.config.WebseedTimeout,
)
return true
```

```
}
```

### 3. 메인 토렌트 루프 통합

```
func (t *torrent) run() {
for {
select {
// ... 기타 케이스들 ...
case  msg := <-t.webseedPieceResultC.ReceiveC():
// WebSeed 피스 결과 처리
t.handleWebseedPieceResult(msg)
case  src := <-t.webseedRetryC:
// WebSeed 재시도 처리
src.Disabled = false
src.LastError = nil
t.startPieceDownloaderForWebseed(src)
// ... 기타 케이스들 ...
}
}
}
```

## 성능 최적화

## 1. 속도 제한

WebSeed 다운로드에 속도 제한을 적용합니다:

```
// 속도 제한 적용
if d.bucket != nil {
waitDuration := d.bucket.Take(readSize)
select {
case <-time.After(waitDuration):
case <-d.closeC:
return false
}
}
```

## 2. 동시 다운로드 제어

```
// 설정에서 최대 동시 다운로드 수 제어
if t.webseedActiveDownloads >= t.session.config.WebseedMaxDownloads {
return false
}

// 최대 WebSeed 소스 수 제한
if len(t.webseedSources) > s.config.WebseedMaxSources {
t.webseedSources = t.webseedSources[:10]
```

```
}
```

### 3. 메모리 효율성

```
// 버퍼 풀 사용으로 메모리 재사용
buf := pool.Get(int(pieces[d.current].Length))

// 사용 완료 후 버퍼 반환
defer buf.Release()
```

## 통계 및 모니터링

### 1. WebSeed 통계

```
// WebSeed 통계 구조체
type Webseed struct {
URL string
Error error
DownloadSpeed int // 바이트/초
}

// WebSeed 목록 조회
func (t *torrent) getWebseeds() []Webseed {
webseeds := make([]Webseed, 0, len(t.webseedSources))
```

```
for _, src := range t.webseedSources {
ws := Webseed{
URL: src.URL,
Error: src.LastError,
DownloadSpeed: int(src.DownloadSpeed.Rate1()),
}
webseeds = append(webseeds, ws)
}
return webseeds
}
```

### 2. 실시간 모니터링

- 각 WebSeed 소스별 다운로드 속도 추적
- 오류 발생 시간과 원인 기록
- 활성 다운로드 수 모니터링
- 전체 다운로드 통계에 WebSeed 기여도 포함

## 관련 문서

### 선행 지식

- 01-system-architecture.md - WebSeed가 전체 시스템에서 차지하는 위치
- 03-01-torrent-lifecycle-process.md - WebSeed 초기화와 관리

## 관련 주제

- 02-data-flow-analysis.md - 피어와 WebSeed의 통합된 다운로드 흐름
- 06-01-performance-optimization-analysis.md - WebSeed 성능 최적화

## 후속 학습

- 06-03-error-handling-recovery-analysis.md - WebSeed 오류 처리 및 복구
- 07-01-library-usage-guide.md - WebSeed 설정 및 사용법

---

> **네비게이션:** 이전: MSE 암호화 분석 | 목차 | 다음: 성능 최적화 분석